

From Traces to Witnessed Contracts: Incremental Precondition–Effect Compilation for LLM Agent Skills

Anonymous Authors

Anonymous manuscript, September 2026

Abstract

Reusable agent skills are increasingly stored as executable action programs, yet those programs rarely carry a machine-checkable account of what each action requires and produces. Consequently, a revision can preserve a plausible final step while silently breaking navigation, possession, or transformation dependencies. We introduce Incremental Witnessed Contract Compilation (IWCC), which compiles successful execution traces into deterministic, provenance-carrying contracts over objective roles, required milestones, precedence, navigation, and producer–consumer dataflow. IWCC screens a complete candidate program before environment execution and reports the violated clause and action position; it neither plans nor repairs. Development uses previously exposed native evidence. We then freeze the compiler, primary comparator, model revisions, statistics, and gates before selecting a zero-overlap confirmation of 60 ALFWorld and ScienceWorld tasks. Four outcome-blind operators produce 240 native replays; together with healthy and naturally surviving programs, the evaluation contains 300 candidates, 202 killed revisions, and 98 controls. IWCC reaches 88.12% balanced accuracy versus 77.48% for the strongest comparator, a 10.64-point gain (task-stratified 95% CI 8.88–12.50 points; paired permutation $p = 1.0 \times 10^{-5}$). Recall is 76.24%, all 98 controls are accepted, and the gain is positive in both environments. Validation takes a median 0.487 ms on one CPU process. These results support a bounded claim: trace-backed structural contracts provide an auditable, low-false-alarm screen for executable skill revisions, but do not certify semantic success or replace native testing.

Keywords: LLM agents; agent skills; specification mining; precondition–effect contracts; regression validation; execution traces

1 Introduction

Language-model agents increasingly retain useful behavior as reusable skills: natural-language procedures, scripts, action programs, and related resources. Programmatic skills can reduce repeated reasoning and improve transfer [5, 13], while structured activation and termination conditions make reuse more reliable [8]. Persistence also changes the failure mode. A malformed program is no longer a single bad trajectory; it becomes a reusable defect that can be called repeatedly, composed with other skills, and copied into later revisions.

Current skill artifacts often expose an objective and an action list but no executable contract connecting them. Consider a household skill that takes a mug, cools it in a refrigerator, and places it in a cabinet. A revised program can still contain the final placement while omitting acquisition, visiting the wrong appliance, or moving the object after possession was lost. Text may remain fluent, and an endpoint-only check may still find every goal keyword. Detecting the defect through native replay is reliable but potentially expensive, stateful, or destructive. An operational library therefore needs a cheap preflight screen that explains why a candidate program is structurally inconsistent before it invokes tools.

Recent work supplies important pieces. Neuro-symbolic skill induction lifts traces into control flow and variable bindings [10]; SKILL-DISCO distills successful paths into reusable finite-state subgraphs [5]; SkillOps represents maintained skills through typed contracts [9]; GraSP uses precondition–effect edges for composition and local repair [14]. Contractual Skills makes governance obligations explicit in skill documents [7], and contract-based drift monitoring checks role-bearing environment assumptions [4]. However, the narrower release-time question remains under-tested: can a compact contract be compiled incrementally from successful native traces, retain evidence for every clause, and distinguish behaviorally killed program revisions from healthy or behaviorally equivalent ones *without executing the candidate*?

We answer this question with Incremental Witnessed Contract Compilation (IWCC). Environment adapters parse commands into typed events. Public objectives bind semantic roles such as subject, destination, transformation device, and measurement target. Successful traces witness required milestones and precedence. A lightweight abstract interpreter then checks possession, location, navigation, and terminal role effects in a complete candidate program. Every learned clause retains hashes of its successful trace witnesses, and set-based merging makes the canonical contract independent of trace arrival order.

The evaluation is deliberately separated into development and confirmation. We use 45 previously exposed healthy trajectories and 32 successful mutation traces to build the final contract and choose baselines. Before opening new outcomes, we freeze the method and a content-blind selection procedure. Confirmation uses 60 previously unused tasks from official ALFWorld validation and ScienceWorld test splits [11, 12]. Deletion, adjacent swap, suffix truncation, and role substitution are specified without outcomes and replayed natively. Natural survivors remain controls. Three instruction models receive exactly the same public program view as IWCC.

This paper makes four contributions:

- a trace-to-contract compiler joining objective roles, milestones, ordering, navigation, and producer–consumer dataflow in one executable preflight artifact;
- provenance and determinism requirements that make every learned clause auditable and invariant to trace insertion order;
- a zero-overlap native confirmation with a frozen primary comparator, three model-family controls, task-clustered inference, and explicit practical gates;
- evidence that the compiled contract improves defect screening while preserving healthy and behaviorally equivalent programs at sub-millisecond median CPU latency.

We ask: **RQ1** Does IWCC improve killed-revision detection over endpoint-only, ordered, trace-similarity, and local language-model validators? **RQ2** Does it preserve valid programs and retain positive effects across environments and mutation operators? **RQ3** Are its decisions auditable and inexpensive enough for pre-execution use?

2 Related Work

Programmatic skill induction. ASI induces and verifies program-based web skills online [13]. NSI lifts traces into logic-grounded programs with explicit control flow and variable binding [10], while SKILL-DISCO represents shared successful structure as parameterized finite-state subgraphs [5]. Skill-Pro defines skills through activation, execution, and termination conditions and verifies candidates through a learning gate [8]. SCALAR iteratively refines symbolic preconditions and effects using grounded reinforcement-learning feedback [15]. These systems learn or execute skills. IWCC instead treats the candidate executable program as fixed and evaluates the contract as a release-time defect detector.

Skill contracts and lifecycle maintenance. SkillOps proposes typed contracts over prerequisites, outcomes, actions, validation, and failures in a library-maintenance layer [9]. Contractual Skills extends human-readable skill packages with permissions, evidence, approval, and output obligations [7]. Contract-based drift monitoring detects changes in role-bearing external assumptions [4]. GraSP uses typed precondition–effect DAGs for orchestration and local repair [14]; SkillFuzz extracts contracts to prioritize risky cross-skill compositions [6]. IWCC is complementary: its unit is an intra-skill action-program revision, its clauses are backed by successful trace hashes, and its endpoint is execution-free discrimination against native behavioral outcomes.

Specification mining. Mining temporal specifications and likely invariants from program executions is established in software engineering [1–3]. Dynamic evidence cannot prove universal correctness; inferred properties are conditioned on observed executions. IWCC adopts that discipline. A clause is a witnessed invariant of a scoped skill family, not a theorem about every future environment. We evaluate false rejection on healthy and native-surviving programs precisely because overfitting positive traces is a central risk.

3 Problem Formulation

Let a task be $x = (e, f, o, s_0)$, where e is the environment, f a skill family, o a public objective, and s_0 the public initial observation. A complete candidate program is $P = (a_1, \dots, a_T)$. Native execution produces success $Y(P, x) \in \{0, 1\}$, but Y is unavailable to a pre-execution validator.

A successful trace $\tau = (x, P, \mathcal{O})$ contains the executed actions and observations. A compiler receives a set $D^+ = \{\tau_i : Y(P_i, x_i) = 1\}$ and emits a family contract $C_{e,f}$. A validator returns

$$V(C, x, P) = (d, L),$$

where $d \in \{\text{valid}, \text{broken}\}$ and L is a list of clause violations with program positions. For a controlled revision, the native label is *killed* when a healthy source program succeeds and the revised program does not. Healthy programs and revised programs that still succeed are controls.

The primary endpoint is balanced accuracy,

$$\text{BA} = \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right),$$

because mutation survival creates unequal class counts. A useful validator must improve BA without buying recall through broad rejection. We therefore predeclare additional gates on recall and false rejection.

4 Incremental Witnessed Contract Compilation

4.1 Typed events and objective roles

An environment adapter normalizes commands into events $g = (\text{op}, u, v)$. Operators include navigation, open/close, acquire, place, clean, cool, heat, focus, measure, and use. Concrete simulator instances (for example, **drawer 4**) remain distinct in state fluents; only objective matching removes instance suffixes. This separation prevents the development error in which two physical receptacles collapsed into one semantic role.

The objective parser binds a goal tuple

$$g = (f, r_s, r_d, r_t, k),$$

containing family, subject role, destination role, optional transformation, and required multiplicity. For open-category ScienceWorld tasks, the subject role is bound by the first qualifying focus event and propagated to acquisition and placement. Thermometer tasks additionally bind a tool role and a branch-dependent colored-box destination.

4.2 Milestones and witnessed clauses

Each successful trace is abstracted to milestone sequence $M(\tau)$, such as **acquire_subject**, **cool_subject**, and **place_subject**. For family f , a milestone is required when its empirical support is at least $\theta_m = 0.8$. A precedence clause $m_i \prec m_j$ is emitted at support $\theta_o = 0.9$ only when the reverse relation is never observed. Successful alternative sequences are retained as branches instead of being labeled anomalous.

For every required or precedence clause c , the contract stores

$$W(c) = \{h(\tau) : \tau \text{ witnesses } c\},$$

where h is SHA-256 over the trace identity, objective, initial observation, and action sequence. ScienceWorld navigation edges are also retained with trace witnesses. The released contract contains 71 required, precedence, and navigation clauses compiled from 77 successful development witnesses.

The compiler deduplicates traces by hash and computes all supports from sets and counters. Consequently, for any permutation π ,

$$\text{compile}(D^+) = \text{compile}(\pi(D^+)).$$

One hundred development permutations produce the same canonical contract hash, and every emitted learned clause has at least one witness.

4.3 Pre-execution abstract interpretation

Validation first checks that all required goal-bound milestones occur, then verifies learned precedence. IWCC adds a single-pass abstract interpreter. It tracks current location and a set of held concrete entities. Acquisition requires the source to equal the current ALFWorld receptacle; transformations require possession and the correct device location; placement requires possession and the current destination. ScienceWorld moves must follow a navigation edge witnessed during development. Violations contain the clause, action, and index.

The validator uses only C, e, f, o, s_0, P . It never sees task identifiers, mutation operators, reference programs, or native outcomes. Complexity is $O(|P| + |C_{e,f}|)$ with small hash-set lookups. It is conservative in a specific sense: acceptance means no modeled structural violation was found, not that the task will succeed.

5 Experimental Design

5.1 Development and method freeze

Development uses previously exposed evidence only: 45 successful native traces and 135 prior mutation outcomes. Thirty-two native-surviving mutations are added as positive witnesses so valid alternative paths are not rejected.

The final development set has 103 killed revisions and 77 controls. IWCC reaches 87.21% BA versus 74.59% for endpoint-only. A first compiler version is retained as negative evidence: collapsing instance names and lacking witnessed ScienceWorld topology limited its gain to 7.77 points.

Before confirmation, we freeze source hashes, thresholds, the contract, the primary comparator, prompt, model revisions, nearest-trace threshold, sampling code, statistical tests, and ten acceptance gates. The first operational lock is retained because its roster request exceeded the five untouched examples remaining in one ALFWorld family. No roster or outcome was produced. A second lock changes only family quotas while preserving 30 tasks per environment and every analytical gate.

5.2 Independent native confirmation

The frozen roster selects 60 task identifiers with zero overlap against all prior evidence: 30 ALFWorld tasks across five reliable families and 30 official ScienceWorld test variations across three families. Selection uses only environment, family, split, compatibility, prior-use exclusion, and a hashed seed order. All 60 expert/gold references succeed.

Four operators are generated before mutation outcomes are observed: deleting a seeded admissible action within the final six actions; swapping it with its successor; truncating the last three actions; and replacing the final objective-bearing destination or device role. Each task receives every operator. A fresh reset and exact initial-state signature precede each native replay. All 240 replays are technically valid; 202 are killed and 38 survive. The evaluation keeps all of them and adds 60 healthy controls, yielding 300 programs and 98 negatives.

5.3 Comparators and model controls

Endpoint-only checks required objective milestones and is the predeclared primary comparator. **Ordered milestones** also checks learned precedence but not state/dataflow. **Nearest successful trace** rejects when normalized edit distance from the nearest successful development operator sequence exceeds 0.6; the threshold is fixed in development. Three deterministic, revision-pinned local judges—Qwen3-4B-Instruct-2507, Phi-4-mini-instruct, and Mistral-7B-Instruct-v0.3—receive the same objective, initial observation, and numbered candidate program. They output valid, broken, or uncertain; as frozen in development, only *broken* triggers rejection. No model sees a healthy reference or private label.

5.4 Statistics and acceptance gates

The candidate program is the scoring unit, while inference respects shared source tasks. We report a two-sided paired sign-permutation test over 60 task-level BA differences using 100,000 frozen-seed draws. A 10,000-draw bootstrap samples tasks within environment–family strata. Confirmation passes only if it contains at least 50 tasks, 100 kills, and 50 controls; IWCC has BA at least 0.80, recall at least 0.75, false rejection at most 0.05, and at least a 10-point gain over endpoint-only; the 95% gain interval excludes zero, $p \leq 0.01$, and both environments improve. The labels are opened once after every method and model prediction is complete.

6 Results

6.1 Primary result

IWCC attains 88.12% balanced accuracy, compared with 77.48% for endpoint-only and ordered milestones. Its 10.64-point gain has a task-stratified 95% interval of 8.88–12.50 points; the paired task permutation test gives $p = 1.0 \times 10^{-5}$. IWCC identifies 154 of 202 killed revisions (76.24% recall) and accepts all 98 controls (100.00% specificity). The Wilson 95% interval for recall is 69.91–81.58%; with zero observed false rejections, the corresponding upper endpoint is 3.77%.

All ten frozen confirmation gates pass. The strongest non-oracle comparator remains endpoint-only. The language-model judges obtain 49.50–51.12% BA: Qwen rejects almost every program, whereas Phi and Mistral trade some specificity for lower recall. These are not claims about their general reasoning quality; they show that an uncalibrated generative judge is a poor substitute for a scoped structural contract under a low-false-alarm requirement.

6.2 Environment and operator effects

The gain is 16.23 points in ALFWorld and 3.41 points in ScienceWorld. Seven of eight family strata are positive and one is tied; none is negative. The weaker ScienceWorld result is important: IWCC reaches only 78.41% BA there, and the aggregate claim should not be read as uniformly strong transfer.

Table 1: Independent confirmation at candidate-program level (percent).

Validator	Balanced accuracy	Recall	Specificity
Endpoint	77.48	54.95	100.00
Ordered	77.48	54.95	100.00
Nearest	50.74	1.49	100.00
Qwen3-4B	49.50	99.01	0.00
Phi-4-mini	50.72	86.14	15.31
Mistral-7B	51.12	60.40	41.84
IWCC	88.12	76.24	100.00

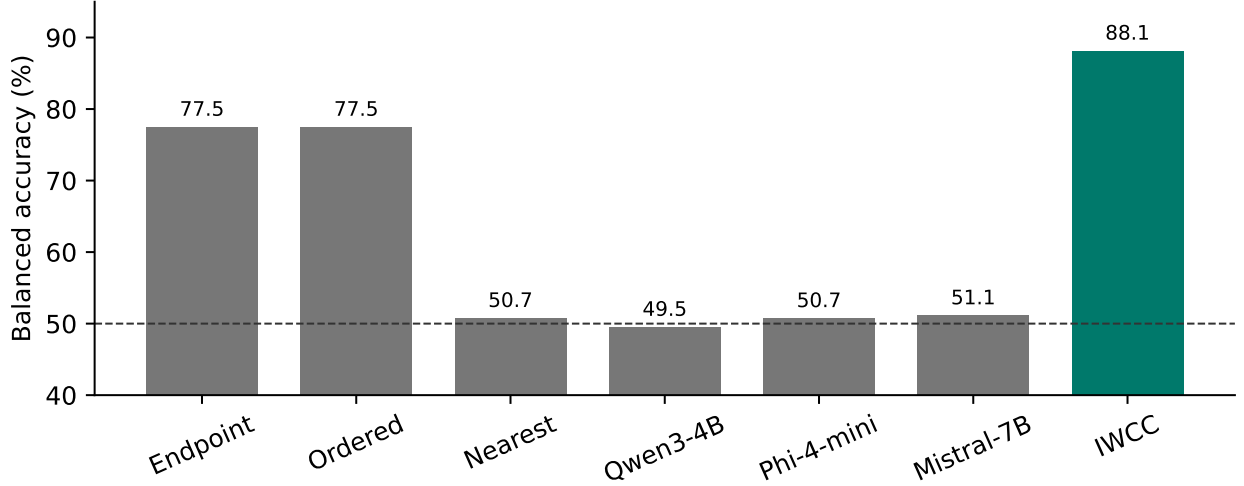


Figure 1: Balanced accuracy on the independent native confirmation. The dashed line is chance-level balanced accuracy; endpoint-only is the strongest comparator.

Killed-revision recall by operator is 72.55% for deletion, 74.19% for adjacent swaps, 80.00% for truncation, and 76.67% for role substitution. Endpoint-only detects 0% of killed adjacent swaps, while IWCC detects 74.19%, isolating the value of intermediate state/dataflow constraints. Role substitution is not solely responsible for the main effect; the three remaining operators still expose the same mechanism.

6.3 Auditability and cost

Among the 154 detected kills, an IWCC violation points exactly to the edited action in 63.64% and within one action in 94.16%. The released contract records witness hashes for learned clauses and a canonical content hash. On a single CPU process, 30,000 post-hoc validation calls have median latency 0.487 ms and throughput approximately 1715 programs/s. In contrast, the three model controls consume 900 GPU inferences and peak at 7.45–13.91 GiB. GPU inference is therefore part of the comparison experiment, not an operational requirement for IWCC.

Table 2: Balanced accuracy by native environment (percent).

Environment	Endpoint	IWCC	Gain
Alfworld	79.39	95.61	16.23
Scienceworld	75.00	78.41	3.41

7 Discussion

Practical use. IWCC fits between skill editing and native regression execution. A rejected program can be returned to a developer with a clause and index; an accepted program proceeds to native tests or a risk-limiting release gate. This ordering avoids tool calls for obvious structural defects but preserves native execution as the semantic oracle. Trace hashes let maintainers inspect why a constraint exists and retire it when its witnesses become obsolete.

Why endpoint checks fail. Endpoint rules recognize that a correct goal action appears somewhere. They cannot tell whether the agent reached the corresponding location, still holds the object, or traversed a supported path. Adjacent swaps preserve action vocabulary and often preserve all milestones, which explains the identical endpoint and ordered aggregate results. Abstract state connects producers to consumers and catches these defects without memorizing full traces.

Relationship to lifecycle governance. Contract compilation answers what a skill promises structurally. It does not decide how much native evidence is enough for release, repair a rejected program, or detect conflicts between multiple skills. Those are distinct lifecycle questions. This separation prevents a single artifact from being credited for planning, verification, repair, and governance simultaneously.

8 Threats to Validity

Construct validity. Controlled operators are not a sample of naturally occurring skill defects. Role substitutions may be easier than semantic logic errors, while some deleted search actions remain behaviorally irrelevant. Native replay labels address whether each edit actually kills the task, and all survivors are retained, but production-repository evaluation remains necessary.

Internal validity. Development evidence is exposed and supports only method selection. Confirmation task identifiers have zero overlap with prior experiments; source, comparator, prompt, statistics, and gates are hashed before outcomes. The quota-only V2 roster lock is disclosed. Validators never access the private label file, as checked by a released information-boundary audit. Deterministic simulator gold paths limit stochastic variance but do not represent LLM policy variance.

External validity. The study covers two text-based simulators, eight confirmation families, and action programs with parsable commands. It does not cover browser DOM changes, source-code agents, arbitrary shell programs, concurrent skills, hidden permissions, or stochastic external APIs. ScienceWorld’s modest gain and the tied non-living family show that trace-backed navigation alone is insufficient for some domains.

Statistical validity. Candidate programs within a task are dependent, so significance is computed on paired task effects and bootstrap resampling is task-stratified. Sixty tasks still give limited family-level precision. The 100,000-draw randomization p-value has a floor near 10^{-5} . The Wilson intervals are descriptive supplements; the frozen bootstrap of the primary gain drives inference.

9 Reproducibility

The artifact releases source, unit tests, both method locks, the zero-overlap roster, 60 healthy trajectories, all 240 mutation specifications and outcomes, public/private candidate separation, 900 model responses, source revisions, hashes, analysis outputs, and both manuscript languages. ALFWorld game files and model weights are referenced by paths/revisions rather than redistributed. Native replay requires the adjacent Paper 01 simulator resources; released-record analysis is CPU-only. The experiment used one RTX 3090 for model controls, Python 3.12, PyTorch 2.8.0, CUDA 12.8, ALFWorld 0.4.2, ScienceWorld 1.2.3, and OpenJDK 17.

10 Conclusion

Executable agent skills need more than plausible action lists. IWCC turns successful traces into deterministic, witnessed structural contracts and uses them to screen revisions before execution. On a frozen, zero-overlap ALFWorld/ScienceWorld confirmation, it improves balanced accuracy by 10.64 points over the strongest comparator, with a positive task-clustered interval and no observed false rejection among 98 controls. The contribution is intentionally bounded: witnessed contracts expose modeled structural defects cheaply and audibly; native execution remains necessary for unmodeled semantics and real-world reliability.

References

- [1] Glenn Ammons, Rastislav Bodik, and James R. Larus. Mining specifications. In *Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 4–16, 2002. doi: 10.1145/503272.503275.
- [2] Ivan Beschastnikh, Yuriy Brun, Sigurd Schneider, Michael Sloan, and Michael D. Ernst. Leveraging existing instrumentation to automatically infer invariant-constrained models. In *Proceedings of the 19th ACM Symposium on Operating Systems Principles*, pages 267–282, 2011. doi: 10.1145/2043556.2043581.
- [3] Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69(1–3):35–45, 2007. doi: 10.1016/j.scico.2007.01.015.
- [4] Linfeng Fan, Yuan Tian, Ziwei Li, and Zhiwu Lu. Skill drift is contract violation: Proactive maintenance for LLM agent skill libraries, 2026.
- [5] Zhongxin Guo, Danrui Qi, Hanwen Gu, Peng Cheng, and Yongqiang Xiong. SKILL-DISCO: Distilling and compiling agent traces into reusable procedural skills, 2026.
- [6] Jinwei Hu, Yi Dong, Youcheng Sun, and Xiaowei Huang. SkillFuzz: Fuzzing skill composition for implicit intents discovery in open skill marketplaces, 2026.
- [7] Ting Liu. Contractual skills: A GovernSpec design framework for enterprise AI agents, 2026.
- [8] Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li, Yisen Wang, Haifeng Zhang, and Jun Wang. Skill-Pro: Learning reusable skills from experience via non-parametric PPO for LLM agents, 2026.
- [9] Hongji Pu, Xinyuan Song, and Liang Zhao. SkillOps: Managing LLM agent skill libraries as self-maintaining software ecosystems, 2026.
- [10] Jie-Jing Shao, Haiyan Yin, Yueming Lyu, Xingrui Yu, Lan-Zhe Guo, Ivor Tsang, James Kwok, and Yu-Feng Li. Lifting traces to logic: Programmatic skill induction with neuro-symbolic learning for long-horizon agentic tasks, 2026.
- [11] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Cote, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. ALFWorld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=0IOX0YcCdTn>.
- [12] Ruoyao Wang, Peter Jansen, Marc-Alexandre Cote, and Prithviraj Ammanabrolu. ScienceWorld: Is your agent smarter than a 5th grader? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 11279–11298, 2022. doi: 10.18653/v1/2022.emnlp-main.775.
- [13] Zora Zhiruo Wang, Apurva Gandhi, Graham Neubig, and Daniel Fried. Inducing programmatic skills for agentic tasks, 2025.
- [14] Tianle Xia, Lingxiang Hu, Yiding Sun, Ming Xu, Lan Xu, Siying Wang, Wei Xu, and Jie Jiang. GraSP: Graph-structured skill compositions for LLM agents, 2026.
- [15] Renos Zabounidis, Yue Wu, Simon Stepputtis, Woojun Kim, Yuanzhi Li, Tom Mitchell, and Katia Sycara. SCALAR: Learning and composing skills through LLM guided symbolic planning and deep RL grounding, 2026.

A Frozen Confirmation Gates

The machine-readable summary records all ten criteria individually. No task or outcome is replaced after selection. All model outputs, including uncertain decisions and wrong rejections, remain in the released files. The manuscript is entered in the completion index only because every frozen field is true.

B Information Boundary

The public candidate record has exactly six fields: candidate identifier, environment, family, objective, initial observation, and program. Task locator, variation, operator, edit index, native score, and killed label appear only in the private file opened by the post-hoc analyzer. Source scanning and run specifications attest that all validators keep the private-label flag false.